
Linked Open Data Application for Movie Knowledge Graph

Nguyen Lam Thai
HUST

Thai.NL261068M@sis.hust.edu.vn

Dinh Trong Nghia
HUST

Nghia.DT261074M@sis.hust.edu.vn

Nguyen Trong Tuan
HUST

Tuan.NT261067M@sis.hust.edu.vn

Vuong Dinh Minh
HUST

Minh.VD261073M@sis.hust.edu.vn

Abstract

tbd

1 Introduction

The rapid growth of digital media has produced a vast amount of structured information about movies, actors, directors, production companies, genres, and ratings. Platforms such as The Movie Database (TMDB) and IMDb maintain millions of records, and this data is widely used for search, recommendation, and analytics. However, it is often scattered across isolated databases and tabular datasets, each with its own schema, identifiers, and access method. As a result, it is difficult to query across sources, to connect semantically related entities or to ask questions that span several entity types. Knowledge graphs address this problem by representing entities and their relationships in an interconnected graph structure, supporting integration, exploration, and reasoning across heterogeneous information sources [Hogan et al., 2021]. This perspective is consistent with the Linked Data principles of using dereferenceable identifiers and links to related resources [Berners-Lee, 2006].

This work develops a Linked Open Data (LOD) application centered on the movie domain. It covers movies and related entities, including people, organizations, genres, countries, languages, and ratings, together with relationships such as cast membership, directorship, production affiliation, and publication date. The resulting knowledge graph is designed as a reusable and interoperable semantic resource, while remaining focused on a representative subset of the domain that can be modeled consistently using RDF, OWL, and Schema.org vocabulary [Schema.org, 2025, W3C RDF Working Group, 2014]. Reusing Schema.org terms rather than inventing new ones lowers the barrier for other applications to interpret the data.

We demonstrate a complete pipeline for building a movie knowledge graph in accordance with LOD principles. As illustrated in Figure 1, the proposed workflow consists of six stages: (1) defining an ontology for the movie domain; (2) collecting relevant movie data from TMDB; (3) transforming the collected CSV data into RDF in Turtle notation using stable URIs; (4) building an OWL ontology that defines the domain classes, properties, and restrictions; (5) establishing links to DBpedia and Wikidata to improve interoperability and achieve a higher level of Linked Open Data integration; and (6) providing a SPARQL interface through Apache Jena Fuseki for querying the resulting graph [W3C SPARQL Working Group, 2013]. As an optional stage, we provide a static web application, hosted on GitHub Pages in the style of DBpedia, to support browsing and exploration of the generated data.

The remainder of this report is organized as follows. Section 2 details our method, defines the ontology for the movie domain, and specifies the evaluation protocol. Section 3 presents the data

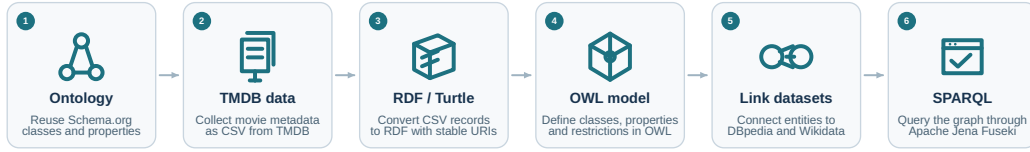


Figure 1: Overview of our method.

collection and transformation. Section 4 introduces the ontology construction and entity linking. Section 5 describes the evaluation. Section 6 discusses limitations and potential future work. Finally, section 7 concludes.

2 Methodology

Figure 1 summarizes the proposed pipeline for constructing and exposing a movie knowledge graph. Movie data are collected from TMDB and stored in CSV files. These tabular records are transformed into Turtle RDF using stable resource URIs. The OWL ontology then adds class, property, and restriction semantics and supports OWL-RL inference. Movie entities are linked to DBpedia and Wikidata using identifiers, titles, release years, and related matching criteria. Apache Jena Fuseki provides a browser-accessible SPARQL endpoint for querying the graph. The optional static web application publishes generated resource pages and ontology views for human browsing.

Here, we describe the first step of the workflow (i.e., defining the ontology for the movie domain) and specify the evaluation protocol, which assesses the resulting knowledge graph by executing representative SPARQL queries. The remaining stages are described in their corresponding sections.

Movie ontology We construct the movie ontology by reusing Schema.org classes and properties to represent movies and their related entities. Table 1 lists all Schema.org terms used in the ontology [Schema.org, 2025]. The classes model movies, people, roles, descriptive entities, organizations, and ratings, while the properties describe movie metadata and relationships, including titles, identifiers, release dates, duration, genres, languages, countries, production companies, ratings, directors, actors, and acting roles. This vocabulary provides a shared semantic model while allowing the data to be serialized as RDF and constrained or extended with OWL.

Evaluation protocol The evaluation protocol queries the graph using SPARQL. The query set covers basic retrieval, such as listing movies, people, and directors, as well as multi-condition questions involving ratings, genres, release years, and cast members. Additional queries test the value of OWL-RL reasoning by asking for inferred relationships, such as a person’s directed movies or a production company’s country. Finally, federated queries retrieve information about a movie from DBpedia or Wikidata through its established links. Queries are executed against the local graph, with reasoning enabled when required, and against the corresponding remote endpoint for external-source questions.

3 Data collection and transformation

3.1 Data collection

The movie dataset is collected from TMDB, a public source of movie metadata. The collection focuses on the first five pages of TMDB’s popular-movie listing, which return 100 movies in total. For each listed movie, detailed metadata and credit information are retrieved. This produces a bounded snapshot of the popular-movie catalogue rather than a complete crawl of TMDB.

The collected movie attributes include the TMDB and IMDb identifiers, title, synopsis, original language, release date, runtime, average vote, and vote count. The collection also records related entities: genres, production companies, production countries, spoken languages, cast members, and

Table 1: Schema.org classes and properties reused in the movie ontology.

Category	Schema.org terms and use
Classes	<code>Movie</code> (movie); <code>CreativeWork</code> (movie superclass); <code>Person</code> (actor, director, or crew member); <code>Role</code> (reified acting role); <code>Genre</code> (movie genre); <code>Country</code> (movie or company country); <code>Language</code> (movie language); <code>Organization</code> (production company); <code>AggregateRating</code> (movie rating); <code>Rating</code> , <code>Thing</code> , and <code>Intangible</code> (reused superclasses).
Object properties	<code>actor</code> (movie or role to actor); <code>aggregateRating</code> (movie to rating); <code>contributor</code> (movie to non-director crew member); <code>countryOfOrigin</code> (movie to country); <code>director</code> (movie to director); <code>genre</code> (movie to genre); <code>inLanguage</code> (movie to language); <code>location</code> (company to country); <code>productionCompany</code> (movie to company); <code>sameAs</code> (movie to an equivalent external resource).
Datatype properties	<code>abstract</code> (movie synopsis); <code>characterName</code> (character portrayed in a role); <code>datePublished</code> (release date); <code>duration</code> (runtime); <code>identifier</code> (TMDB or IMDb identifier); <code>name</code> (entity name); <code>position</code> (cast order); <code>ratingCount</code> (number of votes); <code>ratingValue</code> (rating value); <code>roleName</code> (crew job title).

Table 2: Structure of the collected movie data.

Data group	Information represented
Movie metadata	Identifiers, title, synopsis, language, release date, runtime, and rating information.
Descriptive entities	Genres, production companies, production countries, and spoken languages.
Credits	Cast members, character names, cast order, directors, and person identifiers.
Relationships	Movie-to-genre, movie-to-company, movie-to-country, and movie-to-language associations.

directors. Cast records retain the first ten credited actors, their character names, and their order in the cast. Crew records retain directors and their person identifiers. This selection prioritizes the information needed for movie retrieval, recommendation-style queries, and relationship queries over the entire credit list.

The result is organized as entity tables and relationship tables, under the CSV format. Entity tables store one row per movie, genre, company, country, language, or person reference, while relationship tables preserve many-to-many associations such as movie–genre and movie–production-company links. This separation avoids repeating descriptive values and provides a convenient intermediate representation for RDF generation. Text values are trimmed, missing values are represented as empty fields, and all files are written using UTF-8 encoding.

Table 2 summarizes this organization. The first row contains the central movie records, the second covers reusable descriptive entities, and the third captures the available credit information. The final row distinguishes relationship records from entity records; these links are later converted into RDF predicates between stable resources.

This organization also makes the limitations of the snapshot explicit. Movies can occur more than once in the popular-movie results, and the available metadata depends on the completeness of the source records. In addition, restricting the cast to ten entries reduces the size of the graph but does not represent the full cast of every movie.

3.2 Data transformation

The tabular data are transformed into an RDF graph using Schema.org vocabulary. Each identified entity receives a stable URI composed of a project base URI, an entity type, and its source identifier. For example, movie, person, genre, company, country, and language resources are separated by their respective URI paths. This URI design preserves the identity of entities across files and enables relationships to be represented as links between resources rather than duplicated text values.

Table 3: Qualitative RDF snapshot for “Spider-Man: Brand New Day”.

<pre> <https://example.org/movie/969681> a schema:Movie ; schema:name "Spider-Man: Brand New Day" ; schema:datePublished "2026-07-29"^^xsd:date ; schema:duration "PT2H25M"^^xsd:duration ; schema:aggregateRating [a schema:AggregateRating ; schema:ratingCount 2795 ; schema:ratingValue 7.86] ; schema:genre <https://example.org/genre/12>, <https://example.org/genre/28>, <https://example.org/genre/878> ; schema:director <https://example.org/person/1144604> ; schema:sameAs <https://www.themoviedb.org/movie/969681>, <https://www.imdb.com/title/tt22084616/> . </pre>
--

<pre> <https://example.org/movie/969681> schema:actor [a schema:Role ; schema:actor <https://example.org/person/1136406> ; schema:characterName "Peter Parker / Spider-Man" ; schema:position 0] . </pre>

Movies are represented as instances of `schema:Movie`. Their descriptive fields are mapped to properties such as `schema:name`, `schema:abstract`, `schema:datePublished`, and `schema:duration`. Related entities are represented as typed resources: genres as `schema:Genre`, people as `schema:Person`, production companies as `schema:Organization`, countries as `schema:Country`, and languages as `schema:Language`. Relationship tables consequently become graph edges such as `schema:genre`, `schema:productionCompany`, `schema:countryOfOrigin`, and `schema:inLanguage`.

Values are assigned RDF datatypes where appropriate. Release dates use `xsd:date`, runtimes are represented as `xsd:duration`, rating values use `xsd:decimal`, and vote counts and cast positions use `xsd:integer`. Empty source values are omitted instead of being emitted as empty literals. Ratings are represented by an anonymous `schema:AggregateRating` resource so that the rating value and vote count remain grouped. Similarly, actor information is reified as `schema:Role` resources when character names or cast positions are available.

Table 3 is a qualitative example rather than a complete listing of the resource. The first group of statements describes the movie itself and demonstrates how dates, durations, and ratings are typed. The genre and director objects are URI links to other resources in the graph, whereas the `schema:sameAs` objects link the local resource to its TMDb and IMDb counterparts. The final group shows why a role resource is useful: it preserves both the actor identity and the character name while also retaining the cast position.

4 Ontology constructing and Entity linking

5 Evaluation

6 Discussion

Limitations

Future work

7 Conclusion

References

Tim Berners-Lee. Linked data. <https://www.w3.org/DesignIssues/LinkedData.html>, 2006. Accessed 2026-10-05.

Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4), 2021. doi: 10.1145/3447772.

Schema.org. Getting started with schema.org. <https://schema.org/docs/gs.html>, 2025. Accessed 2026-10-05.

W3C RDF Working Group. Rdf 1.1 concepts and abstract syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation; accessed 2026-10-05.

W3C SPARQL Working Group. Sparql 1.1 query language. <https://www.w3.org/TR/sparql11-query/>, 2013. W3C Recommendation; accessed 2026-10-05.